

WIPRO PYTHON AUTOMATION COURSE

MODULE 2

UNIT TEST FRAMEWORK

Training & Placement Preparation Report

NAME:- Amit Raj

Course:- Python Automation Testing

Technology:- Unittest, PyTest Framework, Page Object Model

Enrolment No. :- 12023052018002

Stream:- CSBS

Semester:- 7th

Date:- 18/06/2026

Summary

This report gives an extensive and practical analysis of Unit 2 - Unit Test Frameworks in the Python Automation Testing course. This unit is split up into three interlinked topics, namely: Python's native unittest framework, PyTest framework and finally, Page Object Model (POM) - a popular design pattern that is used to structure test suites in an automation friendly manner. The report describes concepts, syntax, execution process, fixtures, assertions, test structuring, HTML reporting, framework architecture, base class reuse, CSV file driven data and capturing screenshot in case of a failure along with conversion of Selenium tests to PyTest.

For placement preparation, this report focuses on not just definitions but the logic behind choosing a framework, patterns of automation, examples, debugging and interview questions among other things. This report acts as both a training and revision report.

Table of Contents

1. Introduction
2. Module Scope and Learning Objectives
 - 2.1 Unittest Framework
 - 2.2 PyTest Framework
 - 2.3 Page Object Model
3. End-to-End Automation Framework Architecture
4. Practical Implementation / Mini Project
5. Comparison: Unittest vs PyTest
6. Common Mistakes and Best Practices
7. Conclusion

1. Introduction

Software testing verifies that an application behaves according to its expected requirements. In Python automation, a test framework provides the structure needed to create, execute, organize, reuse, and report automated tests. A framework becomes particularly important when the number of test cases grows and multiple testers or developers work on the same automation project.

The supplied module outline places three concepts together because they form a natural progression. Unittest introduces the basic test lifecycle and assertions; pytest simplifies test development and adds powerful fixtures, selection, parametrization and reporting capabilities; POM provides an architecture for separating test logic from web-page interaction logic.

1.1 Why Unit Test Frameworks Matter in Automation

Repeatability: The same test can be repeated reliably whenever there is a change in the application.

Early fault detection: Small units and reusable parts can be tested before entire processes can be tested.

Regression prevention: Previously functioning functionality can be validated after new code is added.

Maintainability: Reusable fixtures, utilities, and page objects will lead to less repeated code.

Traceability: Assertions and reporting will make faults easy to find and report.

Scalability: A framework structure can expand from a handful of tests to an automated testing suite.

1.2 Relationship Between the Three Topics

Topic	Primary Purpose	Key Skills Developed
Unittest	Understand standard Python test structure	TestCase, test methods, setup/teardown, assertions
PyTest	Build flexible and scalable test suites	Fixtures, markers, discovery, parametrization, reports
POM	Design maintainable Selenium automation	Base page, page classes, inheritance, utilities, screenshots

2. Module Scope and Learning Objectives

The supplied course outline identifies the following topics under Unit 2. They are retained here as the core syllabus and expanded into practical learning objectives.

Section	Topics from the supplied outline	Expected competency
2.1 Unittest	Introduction; first test case; class-level setUp/tearDown; assertions; test suite	Write and execute structured Python tests
2.2 PyTest	Introduction; installation/naming; files/methods/assertions; subsets; fixtures/config; features; multiple tests; Selenium migration; HTML reports	Build and run practical PyTest suites
2.3 POM	Why framework; base/page/test/util/config/screenshot concepts; POM; BasePage and UI concepts; inheritance; CSV; conversion to POM; failure screenshots	Design a reusable Selenium automation framework

2.1 Prerequisites

Python basics: variables, functions, classes, inheritance, exceptions and modules.

Basic OOP concepts: encapsulation, abstraction and inheritance.

Basic Selenium WebDriver knowledge for the POM portion.

Command-line familiarity: running Python modules and installing packages with pip.

Basic HTML/CSS/DOM knowledge and browser developer-tools usage.

2.2 Tools Used in a Typical Practice Environment

Tool / Component	Purpose
Python 3.x	Programming language used for test automation
Unittest	Python standard-library unit testing framework
PyTest	Third-party testing framework used for flexible test execution
Selenium WebDriver	Browser automation for web application testing

pip	Python package installation
IDE / VS Code / PyCharm	Development and debugging
HTML report plugin	Human-readable test execution reports
CSV	Simple external test-data source for practice

3. 2.1 Unittest Framework

Unittest is Python's standard library testing tool. It is an xUnit-based testing tool where tests are usually implemented as subclasses of `unittest.TestCase`. It is a good starting point to learn about test setup, teardown, assertions and suites.

3.1 Unittest Introduction

Built into Python, so no separate installation is required.

Uses `TestCase` classes to group related tests.

Test methods normally begin with `test_` so that the test runner can discover them.

Provides assertion methods such as `assertEqual()`, `assertTrue()`, `assertIn()`, `assertRaises()` and others.

Supports setup and cleanup operations at method and class levels.

Supports test suites and test runners for controlled execution.

3.2 First Test Case

```
import unittest

class TestCalculator(unittest.TestCase):

    def test_addition(self):

        result = 10 + 5

        self.assertEqual(result, 15)

if __name__ == "__main__":

    unittest.main()
```

Execution:

python test_calculator.py

3.3 Understanding the Test Lifecycle

Stage	Method / Concept	Purpose
Class setup	setUpClass()	Runs once before tests in the class
Test setup	setUp()	Runs before each test method
Test	test_*	Contains the actual verification
Test cleanup	tearDown()	Runs after each test method
Class cleanup	tearDownClass()	Runs once after all tests in the class

3.4 Class-Level setUp and tearDown

```
import unittest  
  
class TestLogin(unittest.TestCase):  
  
@classmethod  
def setUpClass(cls):  
print("Open shared test resource")  
  
def setUp(self):  
print("Prepare data before each test")  
  
def test_valid_login(self):  
self.assertTrue(True)  
  
def tearDown(self):  
print("Clean up after each test")  
  
@classmethod  
def tearDownClass(cls):  
print("Close shared test resource")
```

A common placement interview point is the difference between `setUp()` and `setUpClass()`. `setUp()` executes before every test method, whereas `setUpClass()` executes once before the class's tests.

3.5 Assertions

Assertion	Typical use
<code>assertEqual(a, b)</code>	Verify two values are equal
<code>assertNotEqual(a, b)</code>	Verify two values differ
<code>assertTrue(x)</code>	Verify condition is true
<code>assertFalse(x)</code>	Verify condition is false
<code>assertIn(a, b)</code>	Verify a exists in collection/string b
<code>assertIsNone(x)</code>	Verify value is None
<code>assertRaises(Exception)</code>	Verify expected exception is raised

3.6 Creating a Test Suite

```
import unittest

from test_calculator import TestCalculator

from test_login import TestLogin

suite = unittest.TestSuite()

suite.addTests(unittest.defaultTestLoader.loadTestsFromTestCase(TestCalculator))

suite.addTests(unittest.defaultTestLoader.loadTestsFromTestCase(TestLogin))

runner = unittest.TextTestRunner()

runner.run(suite)
```

A test suite is useful when a specific collection of tests must be executed together. In larger modern automation projects, PyTest is often preferred for its simpler discovery and fixture model.

4. 2.2 PyTest Framework

PyTest is a widely used Python testing framework that supports simple function-based tests as well as larger automation suites. The supplied syllabus emphasizes installation, naming

conventions, assertions, subsets of tests, fixtures, configuration, features, multiple test execution, Selenium migration and HTML reports.

4.1 Installation and Naming Conventions

pip install pytest

pytest --version

Test files are conventionally named test_*.py or *_test.py.

Test functions are conventionally named test_*.

Test classes generally use names beginning with Test and should not define an __init__ constructor.

PyTest discovery uses these naming patterns to locate tests automatically.

4.2 Test Files, Test Methods and Assertions

def add(a, b):

return a + b

def test_add_positive_numbers():

assert add(2, 3) == 5

def test_add_negative_numbers():

assert add(-2, -3) == -5

PyTest's plain assert statement is concise and produces useful failure information. The test itself should clearly express the expected behavior.

4.3 Subsets and Test Selection

Run the complete suite with pytest.

Run a specific file with pytest test_login.py.

Run a specific test with pytest test_login.py::test_valid_login.

Use markers to categorize tests, for example smoke, regression or API.

Use -k to select tests by name expression.

Use -m to select tests by marker.

pytest

pytest test_login.py

pytest test_login.py::test_valid_login

pytest -k "login"

pytest -m smoke

4.4 Fixtures and conftest.py

Fixtures are one of PyTest's most important features. A fixture prepares data or resources for tests and can also perform cleanup. Fixtures can be scoped according to how often the resource should be created.

import pytest

@pytest.fixture

def sample_user():

return {"username": "student", "role": "tester"}

def test_user_role(sample_user):

assert sample_user["role"] == "tester"

The conftest.py file is commonly used to define fixtures and hooks that should be available across multiple test files without importing the fixture explicitly.

4.5 Fixture Scopes

Scope	General meaning	Example use
function	New fixture instance for each test function	Test data
class	Shared within a test class	Class-level resource
module	Shared within a Python module	Module resource
package	Shared within a package	Package-level setup
session	Shared for the entire test run	WebDriver / expensive environment setup

4.6 PyTest Features

- Simple function-based tests.
- Powerful fixture dependency injection.
- Markers for categorization and selective execution.
- Parametrization for data-driven tests.
- Automatic test discovery.
- Plugins for reporting and integrations.
- Readable assertion failures.
- Support for parallel execution through appropriate plugins.
- Integration with Selenium and other automation libraries.

4.7 Parametrization

```
import pytest

@pytest.mark.parametrize(
    "a,b,expected",
    [(1, 2, 3), (5, 5, 10), (-1, 1, 0)]
)def test_add(a, b, expected):

    assert a + b == expected
```

Parametrization allows one test function to execute with multiple datasets. It reduces duplicated test code while keeping each input/output combination visible in the test results.

4.8 Running Multiple Test Cases

```
pytest -v

pytest -v tests/

pytest -v tests/test_login.py tests/test_search.py
```

The -v option enables verbose output. A useful practice is to keep tests logically grouped into folders such as tests/, pages/, utils/, data/ and reports/.

4.9 Moving a Selenium Test to PyTest

A common migration is to replace framework-specific unittest setup with PyTest fixtures. The browser driver becomes a reusable dependency rather than being created directly inside every test.

```
import pytest

from selenium import webdriver

@pytest.fixture
def driver():

    browser = webdriver.Chrome()

    browser.maximize_window()

    yield browser

    browser.quit()

def test_homepage_title(driver):

    driver.get("https://example.com")

    assert "Example" in driver.title
```

The yield statement separates fixture setup from teardown: statements before yield prepare the resource, and statements after yield clean it up.

4.10 HTML Reports

A common reporting approach is to install a PyTest HTML reporting plugin and generate an HTML file after execution.

```
pip install pytest-html
```

```
pytest --html=reports/report.html --self-contained-html
```

5. 2.3 Page Object Model

The Page Object Model is a design pattern for UI automation. Instead of placing locators and browser interaction code directly inside test cases, page-specific behavior is encapsulated in page classes. Tests then interact with meaningful methods such as login(), search_product() or checkout().

5.1 Why Have a Framework?

- For avoiding duplications in the browser and locators code.
- For separating test logic from UI implementation.
- For easing maintenance when making any changes to the locators.
- For centralized configuration, test data, logging, and utilities.

- For providing easy setup and teardown.
- For reporting repeatability and debugging.

5.2 Typical Framework Components

Component	Responsibility
Base class / BasePage	Common browser interactions such as click, type, wait and get text
Page classes	Locators and actions belonging to individual application pages
Test classes/functions	Business-level test scenarios and assertions
Utilities	Reusable helpers such as CSV reading, logging and screenshot handling
Configuration	URLs, browser/environment settings and other configurable values
Test data	Input datasets used by tests
Reports	Execution results, logs, screenshots and failure evidence

5.3 What is Page Object Model?

In a POM design, each important page or component is represented by a class. The class exposes actions that reflect what a user can do on that page, while hiding the locator details.

```
from selenium.webdriver.common.by import By
```

```
class LoginPage:
```

```
    USERNAME = (By.ID, "username")
```

```
    PASSWORD = (By.ID, "password")
```

```
    LOGIN_BUTTON = (By.ID, "login")
```

```
    def __init__(self, driver):
```

```
        self.driver = driver
```

```
    def login(self, username, password):
```

```
self.driver.find_element(*self.USERNAME).send_keys(username)
self.driver.find_element(*self.PASSWORD).send_keys(password)
self.driver.find_element(*self.LOGIN_BUTTON).click()
```

5.4 BasePage and UI Concept

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class BasePage:

    def __init__(self, driver):
        self.driver = driver

        self.wait = WebDriverWait(driver, 10)

    def click(self, locator):
        self.wait.until(EC.element_to_be_clickable(locator)).click()

    def type_text(self, locator, text):
        element = self.wait.until(EC.visibility_of_element_located(locator))
        element.clear()
        element.send_keys(text)

    def get_title(self):
        return self.driver.title
```

A BasePage centralizes common Selenium operations. This prevents every page class from re-implementing the same wait-and-click or clear-and-type logic.

5.5 Inheriting BasePage Classes

```
from selenium.webdriver.common.by import By

class LoginPage(BasePage):

    USERNAME = (By.ID, "username")
    PASSWORD = (By.ID, "password")
    LOGIN = (By.ID, "login")
```

```
def login(self, username, password):  
  
    self.type_text(self.USERNAME, username)  
  
    self.type_text(self.PASSWORD, password)  
  
    self.click(self.LOGIN)
```

Inheritance lets a page class reuse common methods from BasePage. This is one of the most important OOP connections in an automation framework.

5.6 Utility to Read CSV Data

```
import csv  
  
def read_csv(path):  
  
    with open(path, newline="", encoding="utf-8") as file:  
  
        return list(csv.DictReader(file))  
  
# Example:  
  
# rows = read_csv("data/login.csv")  
  
# for row in rows:  
  
#     print(row["username"], row["password"])
```

External test data helps separate test logic from input values. CSV is simple and suitable for learning and small datasets; larger projects may use JSON, databases, APIs or other controlled data sources.

5.7 Converting a Test Case to POM Framework

A concrete refactoring could be done in the following four steps:

- Locate pages involved in the test.
- Refactor locators and actions of the page to the corresponding page classes.
- Refactor browser initialization, finalization, and generic code to fixtures or base classes.
- Leave the business logic in the test.

```
# tests/test_login.py  
  
def test_valid_login(driver):  
  
    login_page = LoginPage(driver)
```

```
login_page.login("student", "password")
```

```
assert "Dashboard" in driver.title
```

The test is intentionally short because UI implementation details are hidden inside LoginPage and BasePage.

5.8 Taking Screenshots on Test Failure

```
from pathlib import Path
```

```
def save_screenshot(driver, name):
```

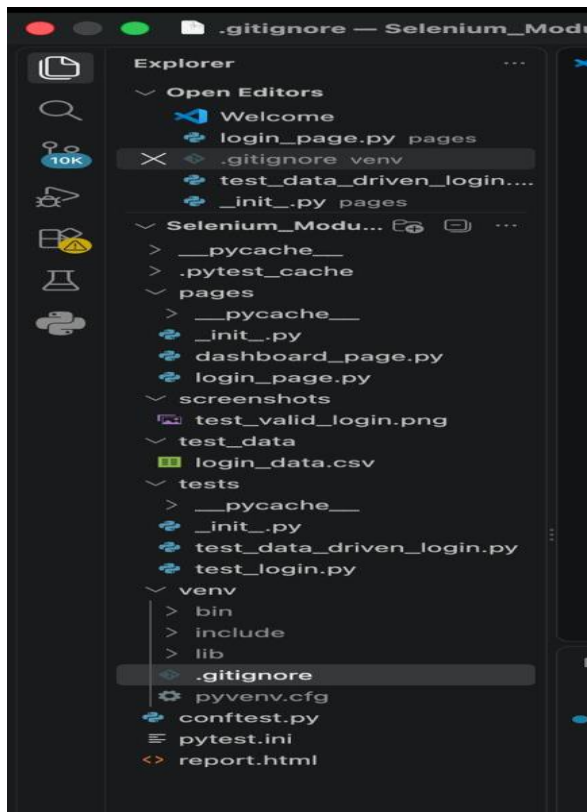
```
    Path("screenshots").mkdir(exist_ok=True)
```

```
    driver.save_screenshot(f"screenshots/{name}.png")
```

In a real PyTest framework, a hook or fixture can trigger screenshot capture when a UI test fails. The screenshot should be named using enough context to identify the test and execution.

6. End-to-End Automation Framework Architecture

A placement-ready practice framework can be organized into separate layers. The exact folder names are shown in this image, but the separation of responsibilities should remain clear.



6.1 Framework Execution Flow

PyTest discovers the selected tests.

Fixtures prepare the environment and WebDriver.

The test creates or receives the required page objects.

Page objects execute UI operations through reusable BasePage methods.

Assertions verify expected application behavior.

On completion, fixtures perform cleanup.

Reports collect pass/fail information; failure evidence can include screenshots and logs.

6.2 Configuration Example

pytest.ini

[pytest]

testpaths = tests

addopts = -v

markers =

smoke: critical smoke tests

regression: regression test cases

6.3 Requirements File

pytest

selenium

pytest-html

Pinning exact package versions is advisable for reproducible projects, especially in team environments. For a training report, the package list above illustrates the main dependencies rather than prescribing versions.

7. Practical Implementation / Mini Project

7.1 Project Title

Web Login Automation using PyTest + Selenium + Page Object Model

7.2 Objective

Automate a representative login workflow using a maintainable framework. The implementation should demonstrate fixtures, page classes, assertions, external data, failure screenshots and HTML reporting.

7.3 Suggested Test Scenarios

ID	Scenario	Expected result
TC01	Valid username and valid password	User is authenticated / redirected to expected page
TC02	Invalid username and valid password	Appropriate validation/error is displayed
TC03	Valid username and invalid password	Appropriate validation/error is displayed
TC04	Blank username and password	Required-field validation is displayed
TC05	Multiple valid/invalid datasets	Each dataset produces the expected result

7.4 Example Fixture

```
# conftest.py

import pytest

from selenium import webdriver

@pytest.fixture

def driver():

    driver = webdriver.Chrome()

    driver.maximize_window()
```

yield driver

driver.quit()

7.5 Example Test with Page Object

def test_valid_login(driver):

driver.get("https://example.com/login")

page = LoginPage(driver)

page.login("valid_user", "valid_password")

assert "Dashboard" in driver.title

7.6 Expected Deliverables

- Clean project structure.
- At least one BasePage and multiple page objects.
- PyTest fixture for WebDriver lifecycle.
- Positive and negative test cases.
- At least one data-driven test.
- HTML execution report.
- Failure screenshot mechanism.
- README containing setup and execution commands.

8. Comparison: Unittest vs PyTest

Aspect	Unittest	PyTest
Availability	Included with Python	Installed separately
Test style	Commonly class-based	Functions or classes
Assertions	self.assert* methods	Plain assert
Setup model	setUp/tearDown methods	Fixtures
Parameterized testing	Available through unittest mechanisms	Very convenient with parametrize
Test discovery	Convention-based runner support	Strong automatic discovery
Plugin ecosystem	More limited	Extensive plugin ecosystem

Learning curve for basic tests	Straightforward if familiar with classes	Very concise for simple tests
Automation framework suitability	Good foundation	Strong fit for scalable Python automation

This table is a technical comparison, not a universal ranking. The appropriate framework depends on project requirements, existing team conventions, integrations and maintenance needs.

9. Common Mistakes and Best Practices

Common mistake	Better practice
Hard-coding the same locator in many tests	Keep locators inside page objects
Using arbitrary sleep() calls everywhere	Use explicit waits and condition-based synchronization
Putting all code in one test file	Separate tests, pages, utilities, data and configuration
Mixing test assertions with low-level locator logic	Keep page actions in page classes and assertions primarily in tests
Creating a new driver in every test without cleanup	Use a fixture with reliable teardown
Ignoring failure evidence	Capture screenshots/logs and generate reports
Using unclear test names	Name tests after the behavior they verify
Duplicating data-heavy tests	Use parametrization or external data
Depending on execution order	Design tests to be independent wherever practical

10.1 Synchronization Best Practice

UI automation frequently interacts with elements that are not immediately ready. Explicit waits should be used to synchronize with meaningful conditions such as visibility or clickability. Fixed sleeps can make a suite slower and may still fail when application timing changes.

10.2 Maintainable Test Design

One test should communicate one clear behavior or scenario.

Use descriptive names and meaningful assertions.

Keep page classes focused on page behavior.

Keep reusable infrastructure in BasePage/utilities/fixtures.

Avoid hidden dependencies between tests.

Use configuration instead of hard-coding environment-specific values.

11. Conclusion

Unit 2 lays down the fundamental knowledge that forms the basis of Python-based automation testing. unittest serves as the base of test cases, assertions, and lifecycles. pytest expands on this knowledge with its simple syntax, fixtures, parametrization, test selection and reporting. After this, Page Object Model will teach you the architecture to build your selenium automation.

In order to prepare yourself for placement, the primary aim should not be learning definitions by heart. You should be capable of creating a small functional project, understanding the purpose of each layer, running tests through command line, understanding the failures, and modifying the framework .

Appendix A – Quick Command Sheet

Command	Purpose
<code>python -m unittest</code>	Run unittest discovery
<code>pytest</code>	Run discovered PyTest tests
<code>pytest -v</code>	Run PyTest execution
<code>pytest tests/</code>	Run tests in a directory
<code>pytest test_file.py::test_name</code>	Run one specific test
<code>pytest -k "login"</code>	Select tests by keyword expression
<code>pytest -m smoke</code>	Run tests marked smoke
<code>pytest --collect-only</code>	Inspect tests discovered by PyTest
<code>pytest --html=reports/report.html --self-contained-html</code>	Generate a self-contained HTML report